

De Roles a Relaciones: La Evolución del Diseño de Autorización

Una guía para arquitectos sobre RBAC, ABAC y ReBAC.



Toda Aplicación Responde a una Pregunta: ¿Quién Puede Hacer Qué?

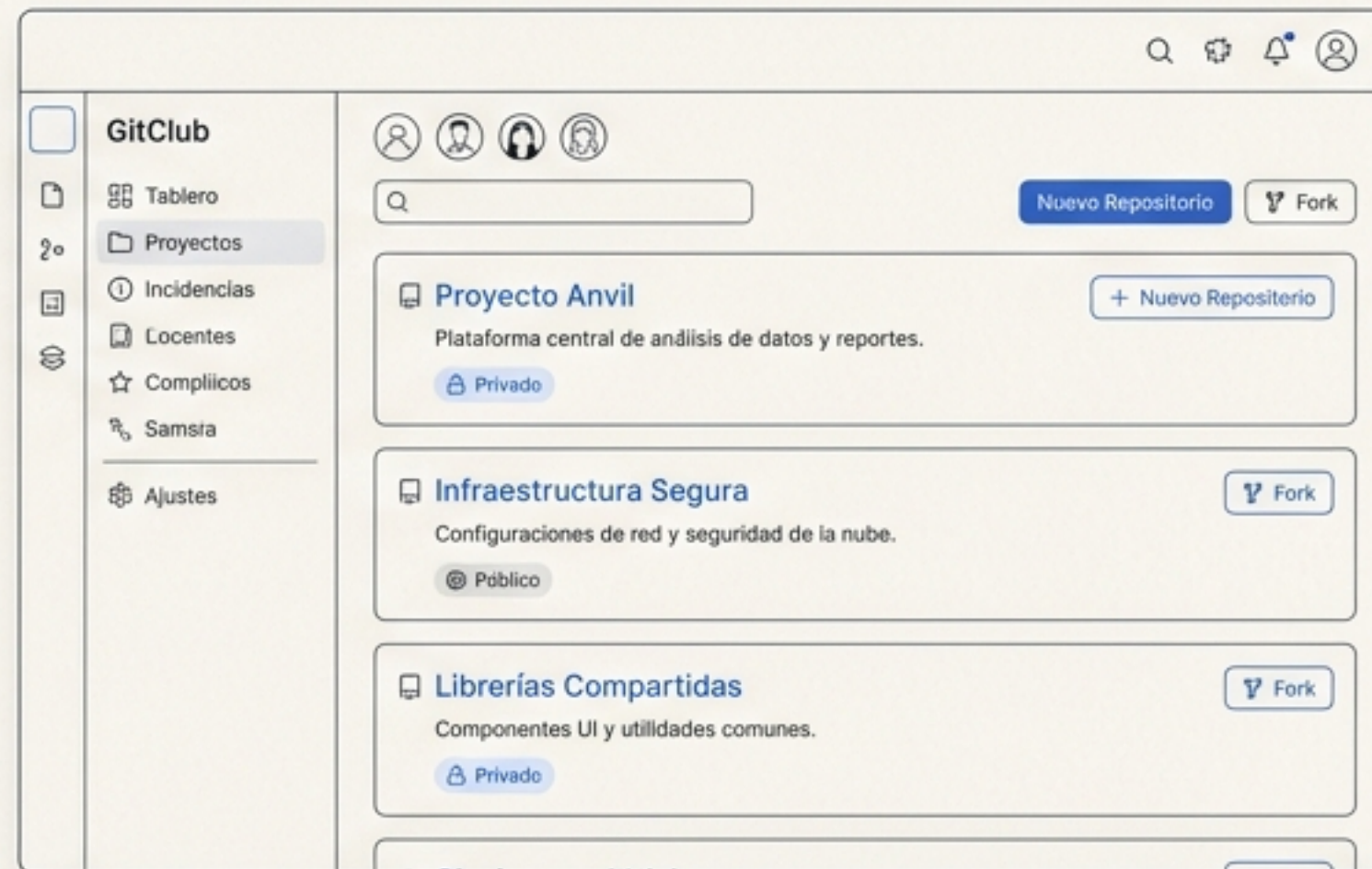
La autorización es el núcleo de la experiencia del usuario. Determina lo que ven, lo que pueden editar y cómo interactúan. Un buen modelo de autorización es intuitivo y flexible; uno malo crea fricción y complejidad.

Introducción a GitClub

A lo largo de esta guía, usaremos nuestra aplicación de ejemplo, GitClub, un análogo de GitHub/GitLab. Su principal desafío de autorización es asegurar el acceso a los recursos, como los repositorios de código.

La Decisión vs. La Ejecución

Nos enfocaremos en *la decisión*: la lógica que responde “¿Tiene este usuario permiso para realizar esta acción en este recurso?”. Un modelo de autorización sólido estructura esta lógica.



El Punto de Partida: Control de Acceso Basado en Roles (RBAC)

RBAC agrupa permisos en *roles* y los asigna a los usuarios. Es simple, intuitivo y funciona bien cuando los permisos se alinean con las funciones organizacionales.

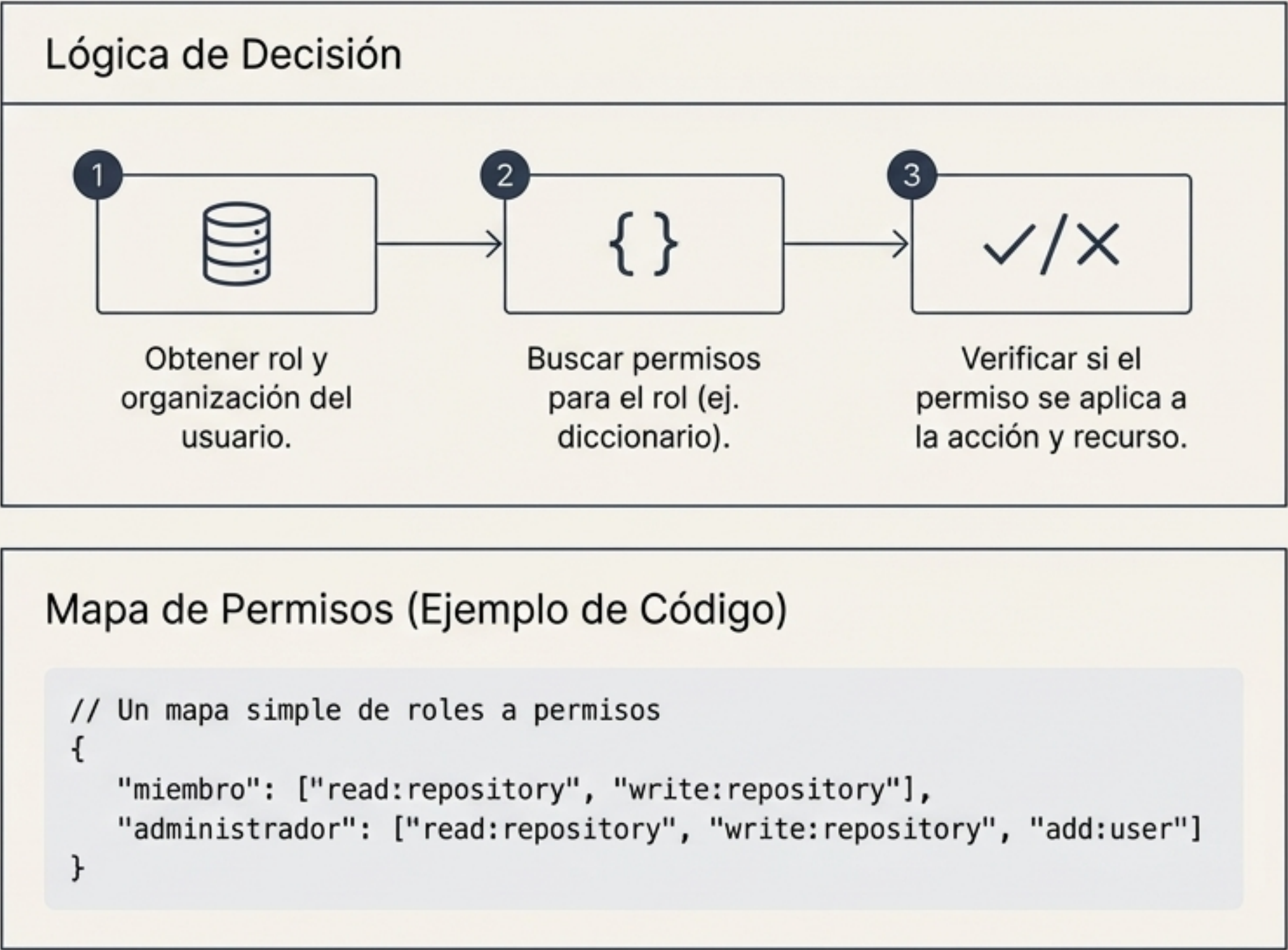
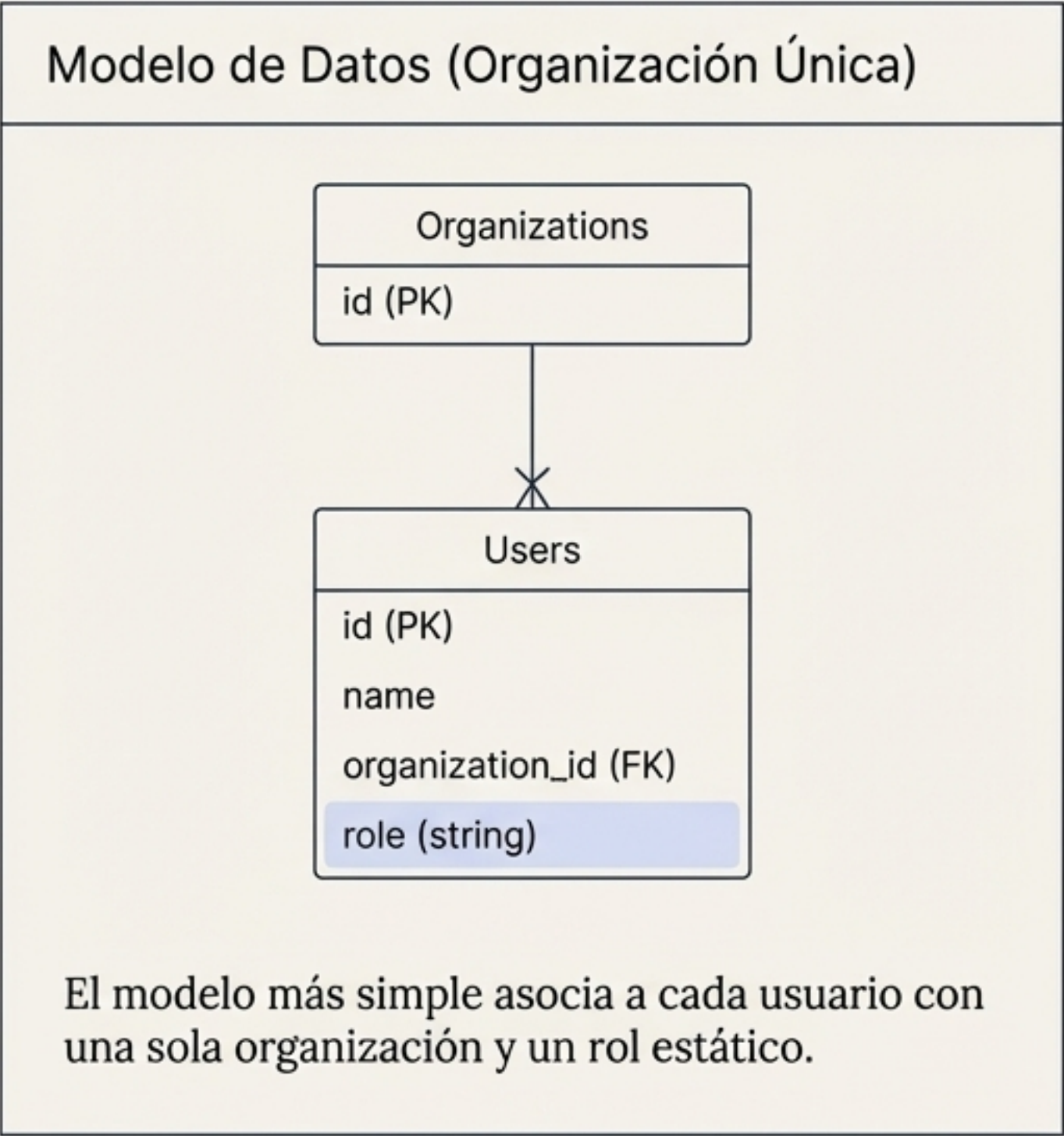


Ejemplo en GitClub (Roles Organizacionales):

- Rol **`miembro`**: Puede leer y escribir en repositorios.
- Rol **`administrador`**: Puede hacer todo lo que hace un `miembro`, y además invitar a otros usuarios a la organización.

En lugar de configurar docenas de permisos individuales, los usuarios eligen entre un número reducido de roles. Esto reduce la carga cognitiva tanto para los desarrolladores como para los usuarios finales.

Implementando RBAC: Estructura y Lógica



Este modelo es fácil de mantener cuando el acceso está determinado únicamente por un rol a nivel de organización.

Los Límites de los Roles: ¿Cuándo se Queda Corto un RBAC Simple?

¿Qué sucede cuando la realidad de la aplicación se vuelve más compleja?

¿Qué sucede cuando la realidad de la aplicación se vuelve más compleja?



Colaboración Inter-Organizacional

Un usuario necesita pertenecer a su organización personal *y colaborar en proyectos de otras organizaciones. El modelo de un rol por usuario ya no funciona.



Permisos Granulares por Recurso

Un cliente necesita que solo ciertos miembros puedan escribir en el repositorio de 'infraestructura', mientras que todos pueden acceder a los demás. Los roles a nivel de organización son demasiado amplios.

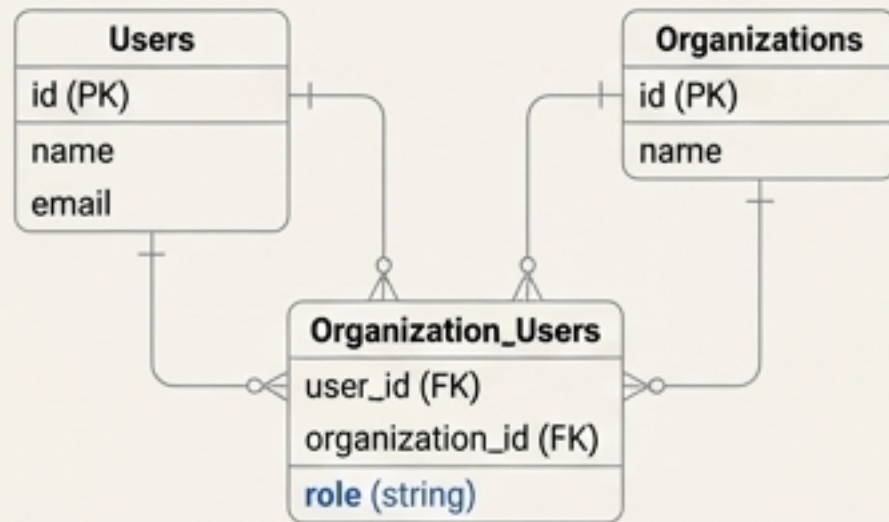


Roles Personalizados

Un cliente empresarial quiere un rol de 'CI/CD' que solo pueda activar flujos de trabajo pero no leer el código. Nuestros roles predefinidos ('miembro', 'administrador') no son suficientes.

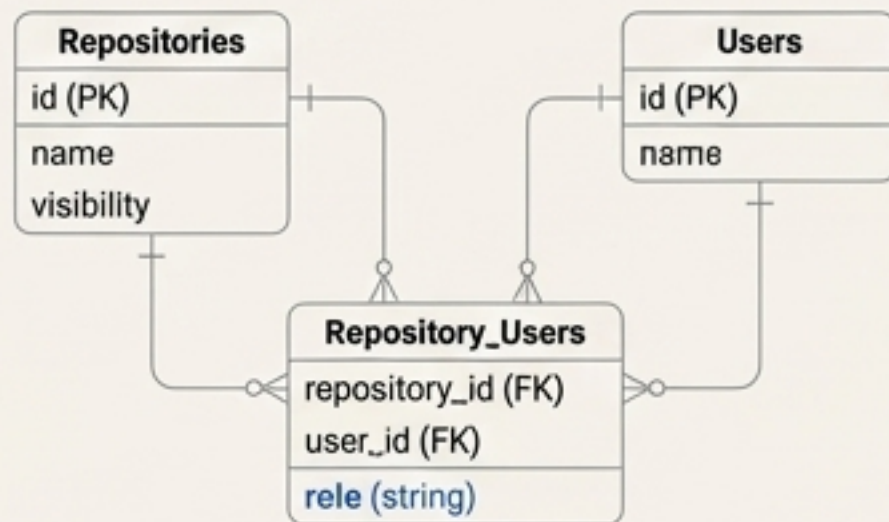
La Evolución de RBAC: Escalando con Complejidad

Solución 1: Roles Inter-Organizacionales



Cambiamos a una relación de muchos-a-muchos. Un usuario ahora puede tener diferentes roles en múltiples organizaciones.

Solución 2: Roles Específicos de Recursos



Además de los roles de organización, introducimos roles a nivel de recurso. Ahora el acceso depende del contexto del repositorio específico.

Conclusión Clave: RBAC es potente y puede adaptarse. Sin embargo, a medida que los requisitos se vuelven más granulares, el número de roles y tablas de unión puede explotar, aumentando la complejidad de la implementación y el mantenimiento.

El Verdadero Punto de Ruptura: Cuando los Roles No Son Suficientes

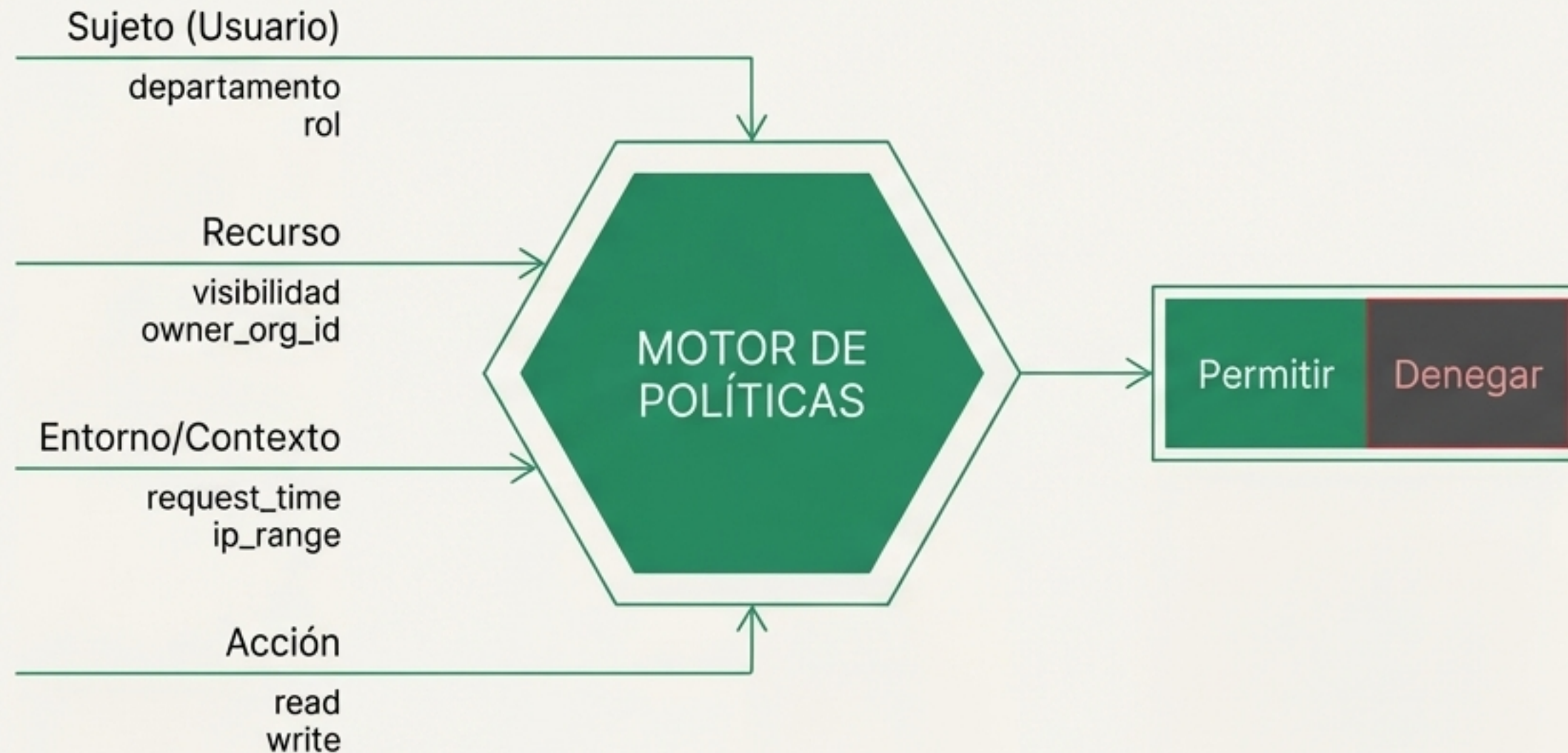
Hay una clase de problemas de autorización que no se basan en **quién eres** (tu rol), sino en **atributos** de la situación.



Forzar estos escenarios en RBAC llevaría a una “explosión de roles” (ej. ``support_engineer_day_shift_trusted_ip``). Necesitamos un modelo más dinámico.

La Siguiente Etapa: Control de Acceso Basado en Atributos (ABAC)

ABAC es un patrón que determina el acceso basándose en atributos dinámicos evaluados en tiempo de ejecución.



Las 4 Categorías de Atributos

1. Sujeto (Usuario):

departamento
clearance_level
rol

2. Recurso:

sensitivity_level
owner_org_id
visibility

3. Entorno/Contexto:

request_time
ip_range
device_trust

4. Acción: El verbo de la política (ej. 'read', 'write', 'merge')

En lugar de preguntar '¿Qué rol tiene este usuario?', preguntamos '¿Los atributos de este usuario, recurso y entorno satisfacen la política para esta acción?'

RBAC vs. ABAC: Una Comparación Directa

Criterios	Control de Acceso Basado en Roles (RBAC)	Control de Acceso Basado en Atributos (ABAC)
Mecanismo	Pertenencia a roles estáticos y predefinidos.	Evaluación de atributos dinámicos en tiempo de ejecución.
Granularidad	Gruesa. Los roles pueden otorgar permisos en exceso.	Fina. Las políticas capturan condiciones precisas, minimizando el acceso.
Flexibilidad	Limitada. No se adapta bien a condiciones como tiempo o ubicación.	Alta. Las políticas se adaptan a cualquier cambio de atributo en tiempo real.
Escalabilidad	Escala pobremente en entornos dinámicos (proliferación de roles).	Escala bien; una política puede gobernar miles de combinaciones.
Caso de Uso Ideal	Roles estables y bien definidos (ej. `admin`, `cajero`).	Entornos dinámicos, complejos y con alto cumplimiento normativo (FinTech, SaaS).

El Próximo Desafío: Cuando el Contexto es una Relación

Algunas reglas de negocio no se expresan bien como atributos aislados, sino como *conexiones* o *relaciones* entre entidades.

Propiedad

Los usuarios pueden eliminar sus *propios* comentarios.

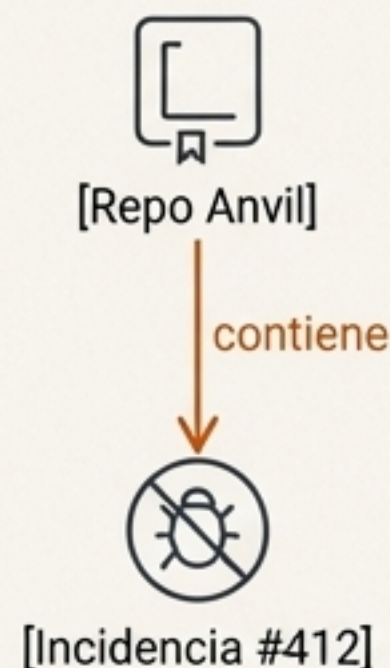
La regla depende de la relación `creado\_por` entre el usuario y el comentario.



Jerarquías (Padre-Hijo)

Puedes leer una incidencia si eres colaborador del *repositorio que la contiene*.

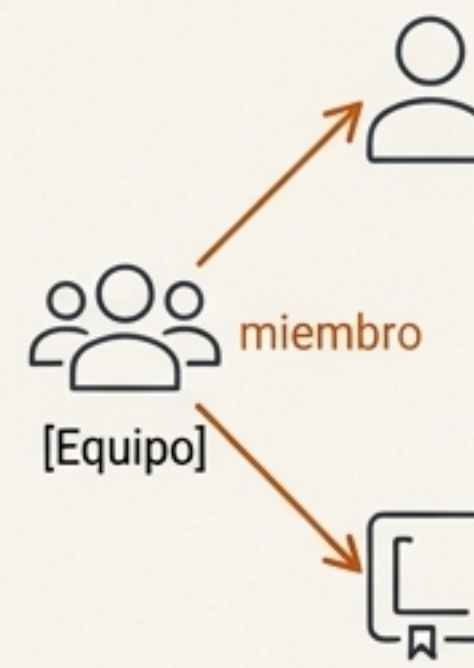
El permiso se hereda a través de la relación `padre`.



Grupos

Eres colaborador de un repositorio si perteneces a un *equipo* que tiene asignado el rol de colaborador en ese repositorio.

El acceso se transmite a través de una membresía de grupo.



Modelar esto como atributos es posible pero torpe. Es más natural pensar en un grafo de relaciones.

La Pieza Final: **Control de Acceso Basado en Relaciones (ReBAC)**

ReBAC organiza los permisos según las relaciones entre los recursos.
Piensa en propiedad, jerarquías, grupos y estructuras de gestión.

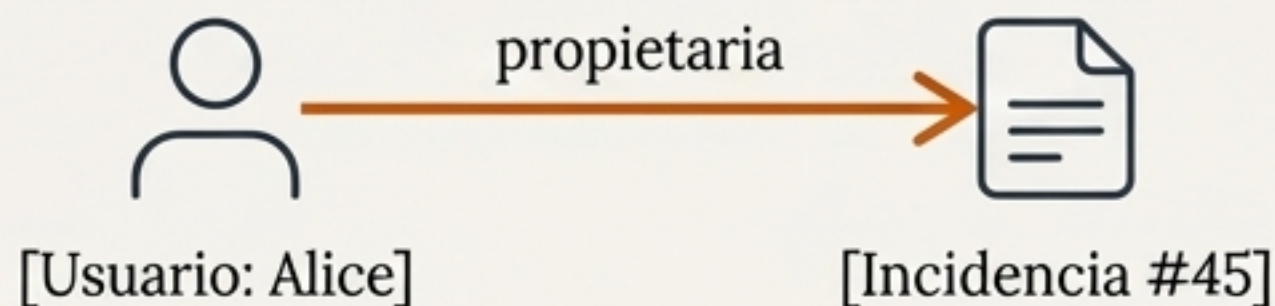
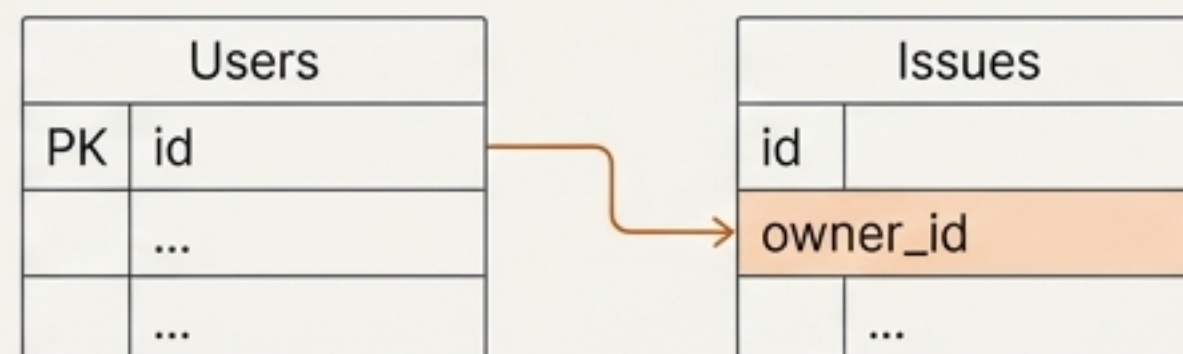
El Poder de ReBAC

ReBAC aprovecha los datos que ya existen en tu aplicación. La columna `owner_id` o `parent_repository_id` no es solo un dato de la aplicación, es un dato de autorización.

Lógica de Decisión (Flujo Conceptual)

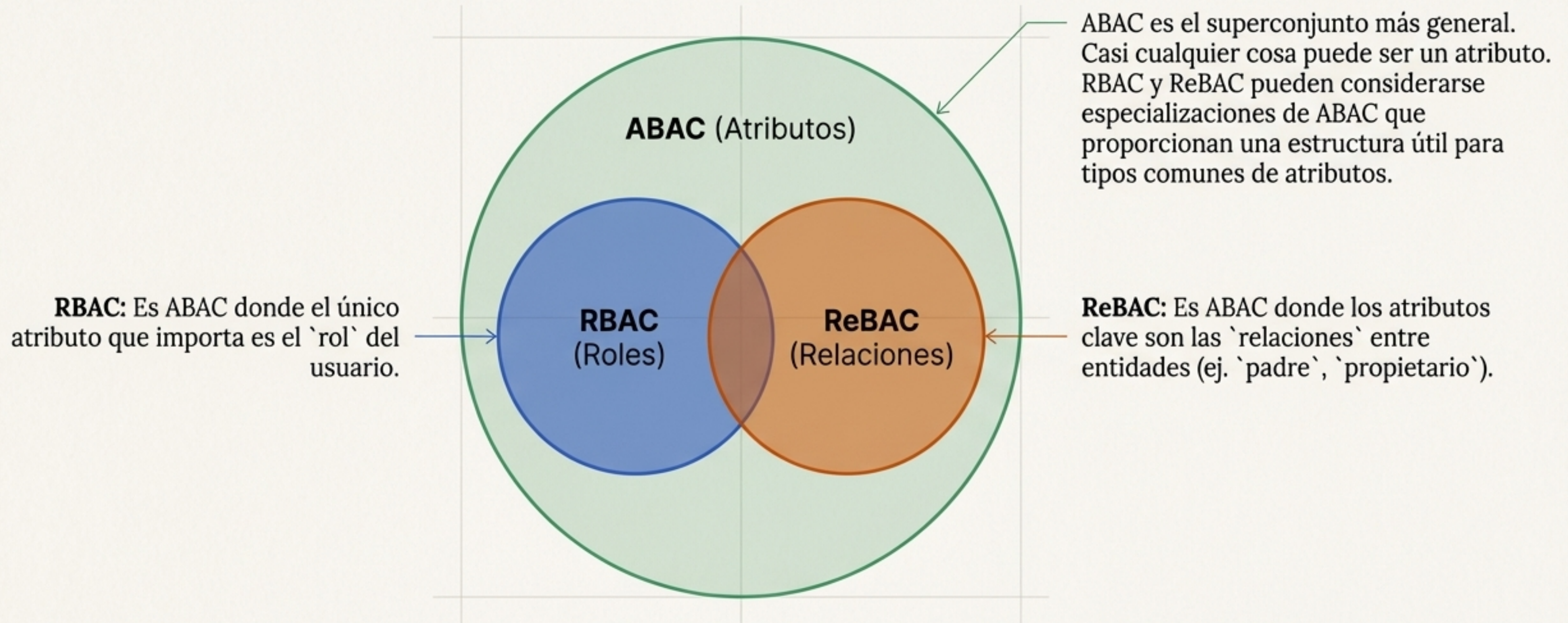
1. ¿El usuario quiere `close` una `issue`?
2. Primero, verificar roles: ¿Es el usuario un `mantenedor` del repositorio padre?
3. Si no, verificar relaciones: ¿Es `issue.owner_id == user.id`?
4. Si alguna es verdadera, permitir.

Ejemplo de Implementación (Propiedad)



El Panorama Completo: Cómo RBAC, ABAC y ReBAC Encajan Juntos

Estos modelos no son mutuamente excluyentes. Son diferentes lentes para ver el mismo problema.



Componiendo Modelos: El Ejemplo de Google Zanzibar

Contexto

Zanzibar es el sistema de autorización global de Google, utilizado por productos como Drive, YouTube y Cloud. Es un sistema ReBAC a hiperescala.

Cómo Funciona (Simplificado)

1. Tuplas de Relación

Todas las relaciones se almacenan como tripletas: `(usuario, relación, objeto)`.

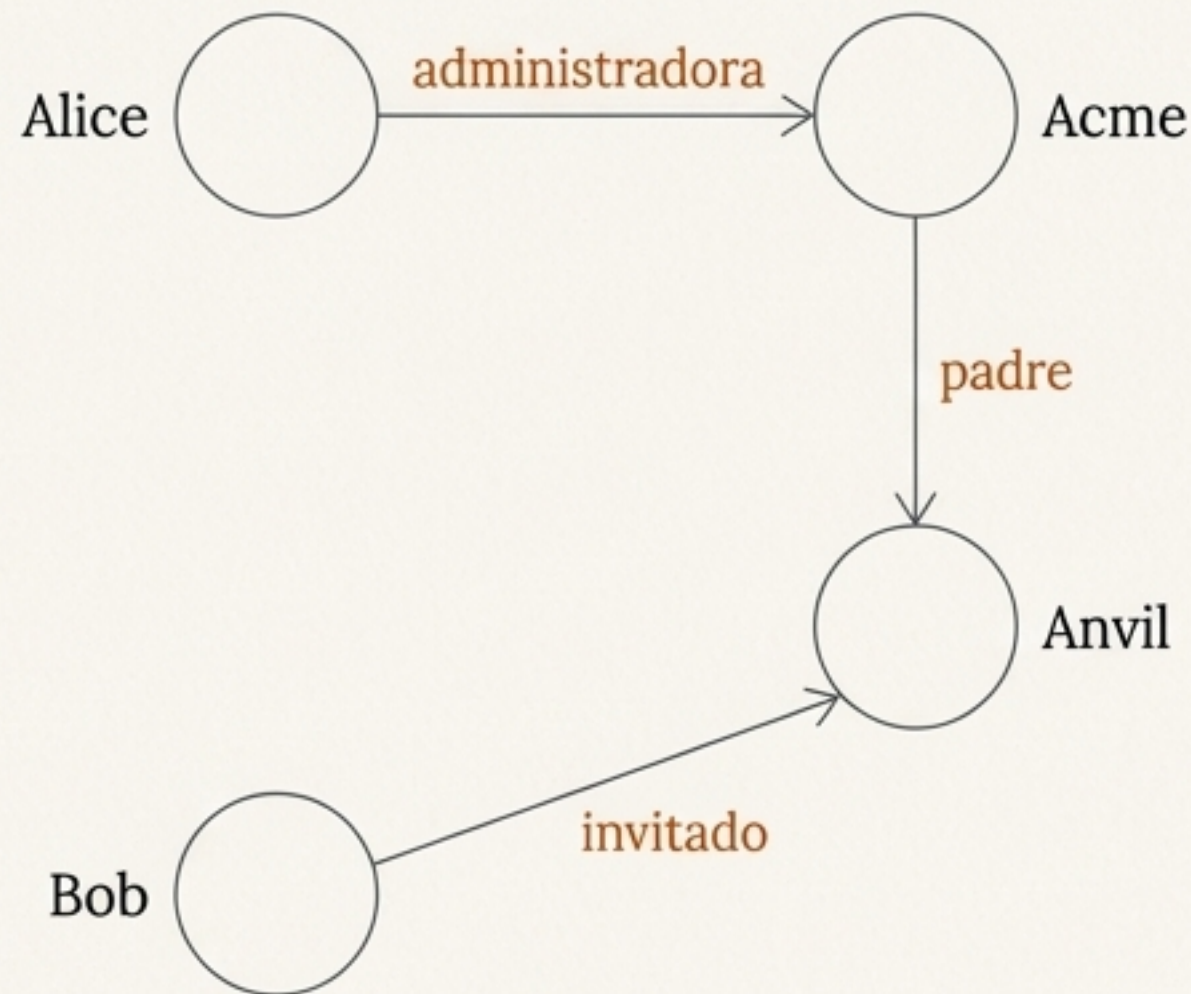
- `(user:alice, 'editor', doc:budget\_q3)`
- `(team:eng, 'miembro', user:bob)`

2. Configuración de Namespace

Lógica que define cómo se heredan o implican las relaciones.

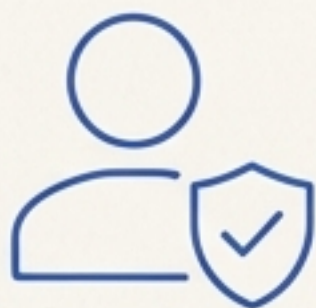
- Los `propietarios` de un documento también son `editores`.
- Los `editores` de una carpeta son `editores` de cualquier documento *dentro* de esa carpeta.

Zanzibar demuestra que un modelo basado en relaciones, cuando se centraliza y se combina con lógica declarativa, puede resolver problemas de autorización a una escala masiva (más de 2 billones de tuplas y 10 millones de QPS).



El Framework de Decisión: ¿Qué Modelo Usar y Cuándo?

Empieza con RBAC si...



- Tus permisos se mapean claramente a roles organizacionales (`admin`, `miembro`, `auditor`).
- Necesitas un modelo simple de entender e implementar.

Ejemplo de GitClub: Asignar roles de `miembro` y `administrador` a nivel de organización.

Introduce ABAC cuando...



- El acceso depende de datos dinámicos o del contexto (tiempo, ubicación, nivel de suscripción).
- Necesitas reglas de grano fino para cumplir con normativas (ej. HIPAA, FinTech).
- Quieres evitar la “explosión de roles”.

Ejemplo de GitClub: Permitir el acceso a un repositorio solo si `is\_public` es `true`.

Aprovecha ReBAC cuando...



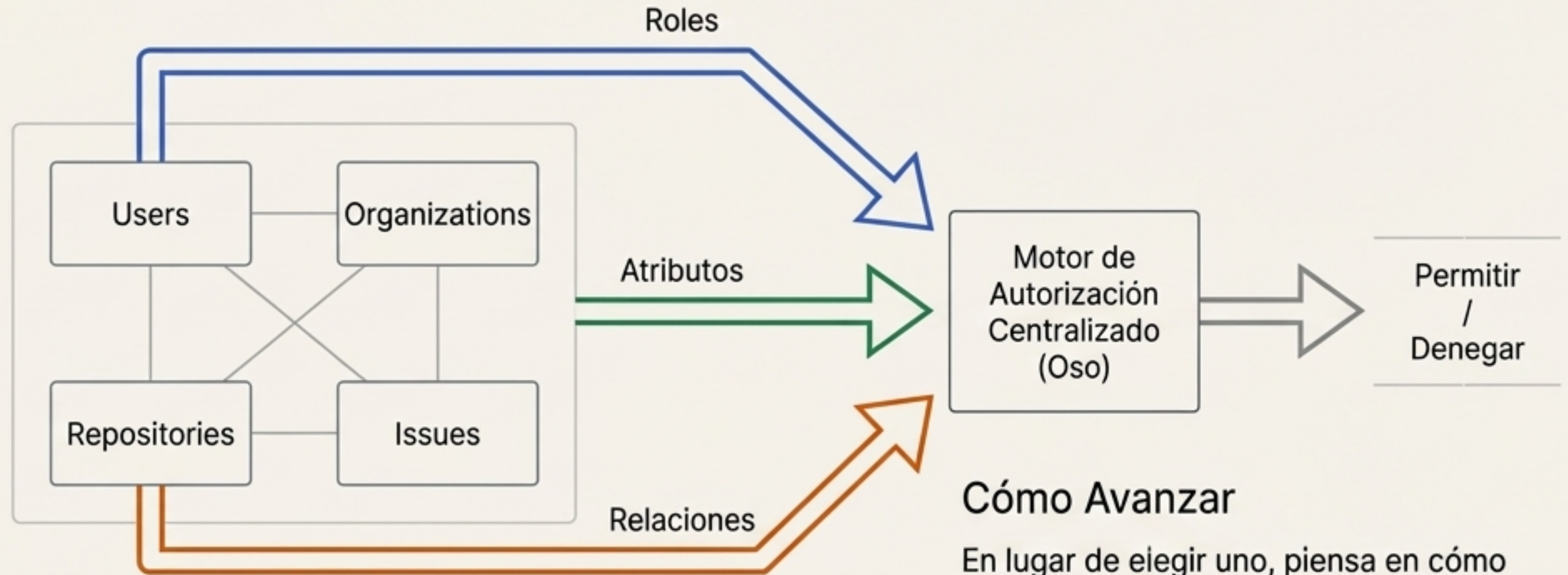
- La autorización se basa en la propiedad, jerarquías o membresías de grupo.
- La lógica de “quién es dueño de qué” o “qué pertenece a qué” es central en tu aplicación.

Ejemplo de GitClub: Permitir que un usuario cierre una incidencia si es el `propietario` de la misma, o si es `mantenedor` del repositorio padre.

La Regla de Oro: Construye la Autorización en Torno a tu Aplicación

Filosofía Central

Tu modelo de autorización debe reflejar la lógica de negocio y los modelos de datos que ya existen en tu aplicación. No fuerces tu aplicación a un modelo de autorización rígido.



El Enfoque Híbrido es la Norma

- Las aplicaciones del mundo real rara vez usan un solo modelo puro.
- Una solución robusta combina la simplicidad de RBAC, la flexibilidad de ABAC y la estructura de ReBAC.

Cómo Avanzar

En lugar de elegir uno, piensa en cómo componerlos. Herramientas como Oso y su lenguaje declarativo Polar están diseñadas para expresar esta lógica compleja sobre tus modelos de datos existentes, permitiéndote evolucionar tu autorización a medida que tu aplicación crece.